

Méthodes et outils pour l'Intelligence Artificielle

Correction Détaillée du Devoir $n^{\circ}1$

Étudiante : N'jie ZAMON

9 Mars 2026

1. Exercice 1 : Programmation Logique (Prolog)

1.1. Création d'un palindrome

Démarche : Un palindrome est une liste qui se lit de la même façon dans les deux sens. Pour transformer $[a, b, c, d]$ en $[a, b, c, d, c, b, a]$, nous devons : 1. Calculer l'inverse de la liste initiale. 2. Extraire la queue de cette liste inversée (pour ne pas doubler l'élément central). 3. Concaténer la liste originale avec ce résultat.

```
1 % Inversion d'une liste
2 inverse([], []).
3 inverse([H|T], L) :- inverse(T, InvT), append(InvT, [H], L).
4
5 % creerPal(ListeEntree, ListeResultat)
6 creerPal([], []).
7 creerPal([X], [X]).
8 creerPal(L, Res) :-
9     inverse(L, [_|QueueInv]), % On inverse et on retire le premier element
10    append(L, QueueInv, Res).
```

Listing 1: Code Prolog pour le palindrome

1.2. Extraction d'un élément sur deux

Démarche : On utilise la récursion sur la structure de la liste. Le prédicat traite la liste par paires $[X, _|T]$. Il conserve le premier élément X , ignore le second, et continue le traitement sur le reste de la liste T .

```
1 % Cas de base : liste vide ou a un seul element
2 extrait([], []).
3 extrait([X], [X]).
4
5 % Cas general : on garde X, on ignore Y et on boucle sur T
6 extrait([X, _|T], [X|Rest]) :-
7     extrait(T, Rest).
```

Listing 2: Code Prolog pour l'extraction

2. Exercice 2 : Logique et Formalisation

2.1. Logique des prédicats

Nous traduisons les phrases F_1, F_2, F_3 en utilisant les quantificateurs universels ($\forall$) et existentiels ($\exists$).

- F_1 : "Certains supporters assistent à tous les matches de coupe."
 $\exists x(S(x) \wedge \forall y(C(y) \Rightarrow A(x, y)))$
- F_2 : "Aucun supporter n'assiste à un match sans enjeux."
 $\neg \exists x \exists y(S(x) \wedge E(y) \wedge A(x, y))$ ou $\forall x \forall y(S(x) \wedge E(y) \Rightarrow \neg A(x, y))$
- F_3 : "Aucun match de coupe n'est sans enjeux."
 $\forall x(C(x) \Rightarrow \neg E(x))$

2.2. Logique des défauts

La logique des défauts permet de représenter des connaissances "normales" qui peuvent être contredites par des faits spécifiques. On utilise la règle de défaut $\frac{\alpha:\beta}{\gamma}$.

Ici, on peut poser que par défaut, un supporter assiste aux matches de coupe :

$$\delta = \frac{S(x) \wedge C(y) : A(x, y)}{A(x, y)}$$

Toutefois, la règle F_2 agit comme une contrainte de cohérence : si on apprend que le match est sans enjeux ($E(y)$), alors $\neg A(x, y)$ devient vrai, ce qui empêche l'application du défaut car $A(x, y)$ ne serait plus cohérent.

2.3. Calcul des extensions

Une extension est un ensemble cohérent maximal de conclusions. **Faits** : $S(\text{Fabrice}), C(\text{Besanon_Marseille}), A(\text{Fabrice}, B_M)$.

1. Par F_3 , comme $C(B_M)$ est vrai, on déduit $\neg E(B_M)$ (Le match a un enjeu).
2. Puisque $\neg E(B_M)$ est vrai, la règle F_2 n'impose pas $\neg A(\text{Fabrice}, B_M)$.
3. L'extension unique contient donc tous les faits initiaux et la conclusion sur l'enjeu du match.

3. Exercice 3 : Ballade dans les étoiles (Heuristique)

3.1. Implémentation des prédicats de base

Nous définissons l'environnement spatial et les fonctions de coût.

```

1 % 1 & 2. Points cardinaux et Coordonnees
2 depart(coruscant).
3 arrivee(kessel).
4
5 planete(alderaan, 4, -1). planete(bespin, -1, -11).
6 planete(coreellia, 3, -2). planete(coruscant, 0, 2).
7 planete(dagobah, 3, -12). planete(dantoine, -1, 13).
8 planete(endor, -5, -11). planete(hoth, -2, -13).
9 planete(kashyyk, 7, 4). planete(kessel, 13, 0).
10 planete(mon_calamari, 12, 8). planete(naboo, 6, -7).
11 planete(ord_mantell, 3, 10). planete(tatooine, 8, -7).
12 planete(yavin, 7, 9).
13
14 % 3 & 4. Graphe de la galaxie
15 voie(coruscant, ord_mantell, 12). voie(coruscant, kashyyk, 9).
16 voie(coruscant, alderaan, 4). voie(coruscant, coreellia, 5).
17 voie(coruscant, endor, 16). voie(ord_mantell, dantoine, 6).

```

```

18 voie(ord_mantell, yavin, 4). voie(yavin, dantoine, 16).
19 voie(yavin, mon_calamari, 7). voie(yavin, kashyyk, 5).
20 voie(kashyyk, mon_calamari, 9). voie(kashyyk, tatooine, 12).
21 voie(mon_calamari, kessel, 9). voie(tatooine, kessel, 9).
22 voie(tatooine, naboo, 2). voie(tatooine, coreellia, 15).
23 voie(naboo, bespin, 6). voie(coreellia, bespin, 12).
24 voie(bespin, hoth, 2). voie(bespin, dagobah, 3).
25 voie(bespin, endor, 4). voie(hoth, endor, 3).
26 voie(hoth, dagobah, 4).
27
28 route(X, Y, D) :- voie(X, Y, D) ; voie(Y, X, D).
29
30 % 5. Coût : Heuristique (Distance Euclidienne)
31 cout(P, H) :-
32     arrivee(But), planete(But, XB, YB),
33     planete(P, XP, YP),
34     H is sqrt((XB-XP)^2 + (YB-YP)^2).
35
36 % 6. Distance parcourue réelle
37 distance([_], 0).
38 distance([P1, P2 | T], D) :- route(P1, P2, D1), distance([P2|T], D2), D is D1 +
    D2.
39
40 % 7. Deplace : Generation des successeurs
41 deplace([Derniere|T], [N, Derniere|T]) :-
42     route(Derniere, N, _), \+ member(N, [Derniere|T]).

```

Listing 3: Prédicats de navigation Star Wars

3.2. Comparaison des algorithmes de recherche

(a) Parcours avec Heuristique (A*)

Démarche : On trie la liste des chemins selon $f(n) = g(n) + h(n)$.

À Coruscant, $h(\text{Coruscant}) \approx 13.1$. Les voisins sont évalués :

- Kashyyk : $g = 9, h(\text{Kashyyk}) = \sqrt{(13 - 7)^2 + (0 - 4)^2} \approx 7.2 \Rightarrow f \approx 16.2$
- Alderaan : $g = 4, h(\text{Alderaan}) = \sqrt{(13 - 4)^2 + (0 - (-1))^2} \approx 9.0 \Rightarrow f \approx 13.0$

Bien que Kashyyk semble plus proche, Alderaan est choisi car son coût total estimé est plus faible au départ. Finalement, le chemin optimal trouvé est : [coruscant, kashyyk, mon_calamari, kessel].

(b) Parcours en Largeur (BFS)

Démarche : Utilisation d'une file (FIFO). On explore tous les chemins de longueur N avant ceux de longueur $N + 1$.

- Niveau 1 : [ord_mantell], [kashyyk], [alderaan], [coreellia], [endor].
- Niveau 2 : [kashyyk, mon_calamari], [tatooine, kashyyk], etc.
- Niveau 3 : **[coruscant, kashyyk, mon_calamari, kessel]** est trouvé car il possède le minimum de segments (3).

(c) Parcours en Profondeur (DFS)

Démarche : Utilisation d'une pile (LIFO). L'ordre dépend de l'ordre d'écriture des faits dans la base.

Si ord_mantell est le premier fait, l'algo ira vers dantoine, puis yavin, mon_calamari et enfin kessel. Le chemin est : [coruscant, ord_mantell, dantoine, yavin, mon_calamari, kessel]. C'est un chemin sous-optimal.